Not Reading, Not Misreading: Where Thai OCR Vision-Language Models Lose Vowel and Tone Marks

ธนกฤต เสมอหน้า[1] ธนวัฒน์ สุริยะ[2] ผศ.ดร. นนท์ คนึงสุขเกษม[3]

คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

เลขที่ 1 ซอยฉลองกรุง 1 แขวงลาดกระบัง เขตลาดกระบัง กรุงเทพมหานคร 10520

[1]67070060@kmitl.ac.th [2]67070236@kmitl.ac.th [3]nont@it.kmitl.ac.th

บทคัดย่อ

Vision-Language Model (VLM) แบบเปิดสำหรับ OCR ภาษาไทยอ่านหน้าเอกสารได้ถูกต้องเกือบทั้งหน้าแล้ว แต่ยังมีคำถามที่สำคัญในทางปฏิบัติอยู่ข้อหนึ่ง คือ สระและวรรณยุกต์ของไทยมีขนาดเล็กและเครื่องหมายเพียงตัวเดียวก็เปลี่ยนความหมายของคำได้ งานนี้จึงถามว่าเครื่องหมายเหล่านี้หายไปที่ใด ไม่ได้ถามเพียงว่าหายไปเท่าไร เราเสนอคะแนนที่วัดเครื่องหมายโดยไม่หักคะแนนเรื่องลำดับการอ่าน หักคะแนนข้อความที่เขียนเกิน และไม่ให้คะแนนกับข้อความที่ซ้ำจากการวนซ้ำ บนชุด calibration ของ ThaiOCRBench โมเดล Typhoon OCR 1.5 (2B) เขียนวรรณยุกต์ผิดเพียง 0.65% ในตำแหน่งที่อ่านพยัญชนะฐานถูกต้อง และเมื่อให้ข้อความก่อนหน้าที่ถูกต้อง โมเดลเลือกวรรณยุกต์ถูกต้อง 98.8% แม้ไม่เห็นภาพ เครื่องหมายที่หายไปส่วนใหญ่ไม่ได้ถูกอ่านเลย กล่าวคือ บรรทัดหายไปทั้งบรรทัด ข้อความขาดเป็นช่วง หรือ output ถูกการวนซ้ำกินไป ในบรรดาวิธี decode ที่ไม่ต้อง train เพิ่ม การหยุดเมื่อตรวจพบการวนซ้ำไม่เปลี่ยน output ที่ไม่วนซ้ำเลยและเพิ่มคะแนน mark F1 ขึ้น 0.8 จุดสำหรับ Typhoon และ 10.8 จุดสำหรับโมเดลไทยขนาด 0.9B ส่วน repetition penalty ทำให้ F1 ของหน้าที่ไม่วนซ้ำลดลง และการหักความรู้ภาษาออกทำให้วรรณยุกต์ที่ถูกอยู่แล้วหายไป **[ผลของโมเดลอื่นและการยืนยันบนชุดข้อมูลที่เก็บไว้ (locked split) ยังไม่เสร็จ]** ผลทั้งหมดเป็นผลเบื้องต้น ใช้ benchmark เดียว ชุดข้อมูล calibration และกฎบางข้อกำหนดหลังจากเห็นข้อมูลแล้ว

คำสำคัญ—OCR ภาษาไทย, vision-language model, วรรณยุกต์และสระ, การวนซ้ำ, การ decode โดยไม่ต้อง train เพิ่ม

1. Introduction

อักษรไทยเขียนพยัญชนะเป็นแนวหลัก แล้วมีเครื่องหมายซ้อนอยู่ด้านบนและด้านล่าง ได้แก่ สระบน สระล่าง และวรรณยุกต์สี่รูป เครื่องหมายเพียงตัวเดียวเปลี่ยนทั้งความหมายและเสียงของคำได้ (“ข่าว” กับ “ข้าว”) การถอดข้อความที่ทำเครื่องหมายหายหรือสลับผิดตัวจึงผิดสำหรับผู้อ่าน แม้ตัวอักษรเกือบทั้งหมดจะถูกต้อง ทว่าการประเมิน OCR ภาษาไทยในปัจจุบันใช้ตัวชี้วัดระดับข้อความทั้งก้อน ได้แก่ ความทับซ้อนของ n-gram และ normalized edit distance ใน ThaiOCRBench [1] และ character error rate ในงานอื่น ตัวเลขเหล่านี้บอกไม่

ได้ว่าปัญหาอยู่ที่เครื่องหมายหรือไม่ และคะแนนที่เทียบทั้งลำดับยังลงโทษโมเดลที่อ่านคอลัมน์ของหน้าในลำดับต่างจากเฉลยด้วย

ปัจจุบันมี VLM แบบเปิดที่ train มาเพื่อ OCR ภาษาไทยแล้ว [2–4] เมื่อโมเดลเหล่านี้ทำเครื่องหมายผิด ผิดเพราะอะไร หลักฐานในภาพของเครื่องหมายขนาดเล็กอาจอ่อนหรือหายไป ความรู้ภาษาของโมเดลอาจทับหลักฐานในภาพ หรือโมเดลอาจไม่ได้อ่านเครื่องหมายนั้นเลย เพราะข้ามบรรทัด หาข้อความไม่เจอ หรือติดการวนซ้ำ สาเหตุแต่ละทางต้องแก้ต่างกัน ถ้าปัญหาอยู่ที่ภาพ ต้องอ่านซ้ำหรือขยายภาพ ถ้าปัญหาอยู่ที่ความรู้ภาษา ต้องหักความรู้ภาษาออกแบบ contrastive decoding [5, 6] และถ้าข้อความไม่ถูกอ่านเลย ต้องแก้ที่วิธี decode หรือการหาตำแหน่งข้อความ

เราศึกษาคำถามนี้บน ThaiOCRBench กับ Typhoon OCR 1.5 และ checkpoint แบบเปิดอีกสี่ตัว ในงานอ่านทั้งหน้าและงานอ่านข้อความสั้น งานนี้มีส่วนร่วมสามข้อ

1. **คะแนนเครื่องหมายที่ไม่หักเรื่องลำดับการอ่าน** (หัวข้อ 3) ให้คะแนนสระบน สระล่าง และวรรณยุกต์ทีละบรรทัดของเฉลย ไม่ว่าบรรทัดนั้นจะอยู่ตรงไหนของ output หักคะแนนข้อความที่เขียนเกิน และไม่ให้คะแนนกับสำเนาที่ซ้ำจากการวนซ้ำ

2. **การวินิจฉัยว่าเครื่องหมายหายไปที่ใด** (หัวข้อ 5) หลังจากอ่านพยัญชนะถูก Typhoon เขียนวรรณยุกต์ผิดเพียง 0.65% และเมื่อให้ข้อความก่อนหน้าที่ถูกต้อง โมเดลเลือกวรรณยุกต์ถูก 98.8% แม้ไม่เห็นภาพ เครื่องหมายที่หายส่วนใหญ่ไม่ได้ถูกอ่านเลย **[ผลวินิจฉัยเดียวกันกับโมเดลอื่น]**

3. **ผลของวิธี decode ที่ไม่ต้อง train ต่อเครื่องหมาย** (หัวข้อ 6) การหยุดเมื่อตรวจพบการวนซ้ำไม่เปลี่ยน output ที่ไม่วนซ้ำเลยและเพิ่มคะแนน mark F1 repetition penalty ทำให้ F1 ของหน้าที่ไม่วนซ้ำลดลง และการหักความรู้ภาษาออกทำให้วรรณยุกต์ที่ถูกอยู่แล้วหายไป **[การยืนยันบน locked split]**

สำหรับโมเดลที่ศึกษา ความผิดของเครื่องหมายที่ยังเหลือในโมเดล OCR ภาษาไทยที่เก่งอยู่แล้วเกิดขึ้นก่อนถึงตัวเครื่องหมาย จุดที่แก้ได้จริงจึงอยู่ที่วิธี decode และการหาข้อความให้เจอ ไม่ใช่การทำให้มองวรรณยุกต์ชัดขึ้น

2. Related Work

OCR ภาษาไทยด้วย VLM ThaiOCRBench [1] ประเมิน VLM ทั้งแบบปิดและแบบเปิดบนงานเอกสารภาษาไทย รวมถึงงานอ่านทั้งหน้าและงานอ่านข้อความสั้น โดยใช้ตัวชี้วัดแบบความทับซ้อนของ n-gram และ edit distance (BLEU, METEOR, F1 และ normalized Levenshtein ส่วนงานอ่านข้อความสั้นใช้ ANLS) บทนำของงานระบุว่า diacritic ที่ซ้อนกันเป็นความยากของภาษาไทย แต่ไม่มีคะแนนที่แยกเครื่องหมายออกมา หลังจากนั้นมี OCR ภาษาไทยแบบเปิดออกมาหลายตัว ได้แก่ Typhoon OCR [2] เวอร์ชัน 1.5 ซึ่งเป็น full fine-tune ของ Qwen3-VL-2B [7] และผู้เขียนระบุว่า infographic เป็นจุดที่ยังต้องปรับปรุง Pathumma-llm-vision 3.0 [3] ซึ่งเป็น fine-tune ของ Qwen3-VL-2B อีกตัว และ wayu-paxa-ocr-zero [4] ซึ่งเป็น fine-tune ของ PaddleOCR-VL [8] ด้วยหน้าเอกสารไทยสังเคราะห์ ทุกงานรายงานความแม่นยำรวม ส่วนงานของเราถามว่าเครื่องหมายหายไปที่ใด

Diacritic ใน OCR อักษรที่มี diacritic มากเป็นที่รู้กันว่ายากสำหรับ OCR มีงานเสนอการแก้ข้อความหลัง OCR สำหรับ diacritic ภาษาเวียดนาม [9] และงานประเมิน OCR ภาษาอาหรับแยกข้อความที่มี diacritic เป็นเงื่อนไขต่างหาก [10] เราไม่พบงานที่ให้คะแนนเครื่องหมายภาษาไทยแยกจากลำดับการอ่านและข้อความที่เขียนเกิน

การวนซ้ำและการตกหล่น การ decode ที่เลือกคำตามความน่าจะเป็นสูงสุดทำให้ข้อความวนซ้ำ [11] และโมเดล OCR เอกสารก็เสื่อมในแบบเดียวกัน Nougat รายงานหน้าที่โมเดลพิมพ์ประโยคเดิมซ้ำและหยุด generate เมื่อสัญญาณความแปรปรวนของ logit ลดลง [12] olmOCR สุ่มใหม่ด้วย temperature ที่สูงขึ้น [13] pipeline ของ PaddleOCR-VL ตัดข้อความซ้ำหลัง decode เสร็จ [14] และโค้ดที่ DeepSeek-OCR เผยแพร่ห้าม n-gram ซ้ำภายในหน้าต่างหนึ่ง [15, 16] CC-OCR วัดการวนซ้ำในฐานะ hallucination แบบหนึ่งของ OCR [17] งานเหล่านี้มองการวนซ้ำเป็นสิ่งที่ต้องตรวจจับหรือหลีกเลี่ยง ส่วนเราวัดว่าวิธีแก้แต่ละแบบทำให้เครื่องหมายในหน้าที่ไม่วนซ้ำเสียไปเท่าไร รวมถึง repetition penalty ของ CTRL [18] ที่ package OCR บางตัวเปิดไว้เป็นค่าเริ่มต้น

ความรู้ภาษาและ decoding ที่ไม่ต้อง train Contrastive decoding ลด hallucination ของ VLM โดยลดน้ำหนักสิ่งที่โมเดลจะทำนายเมื่อไม่มีภาพหรือภาพถูกทำให้เสีย ได้แก่ VCD [5], M3ID [6] และ PAI [19] วิธีเหล่านี้ตั้งสมมติฐานว่าความรู้ภาษาเป็นต้นเหตุ แต่สำหรับวรรณยุกต์ไทยเราพบว่าความรู้ภาษามักถูกอยู่แล้ว วิธีที่เราใช้เป็นการเขียนใหม่ จึงเรียกว่า “M3ID-form” และ “PAI-style” ส่วน speculative decoding [20] ซึ่งในงานนี้ใช้ prompt lookup เป็นตัวร่าง [21] ทำให้ decode เร็วขึ้นโดย output ไม่เปลี่ยนเมื่อคำนวณแบบแม่นยำ

3. Scoring Marks Without Charging Reading Order

เครื่องหมายที่เราวัด (เรียกว่า *mark*) ได้แก่ สระบนหกตัว (กั กิ กี กึ กื กั์) สระล่างสองตัว (กุ กู) และวรรณยุกต์สี่รูป (ก่ ก้ ก๊ ก๋) เฉลยของหน้าเต็มมีหลายบรรทัด และโมเดลอาจอ่านคอลัมน์ของหน้าในลำดับต่างจากเฉลย การเทียบแบบ global alignment จึงนับ mark ที่อ่านถูกว่าผิด

นิยาม เราประมวลผลบรรทัดของเฉลยที่ยาวอย่างน้อยแปดตัวอักษร เรียงจากบรรทัดยาวไปสั้น แต่ละบรรทัดหาช่วงของ output ที่ตรงกับบรรทัดนั้นด้วย character error rate ต่ำกว่า 40% โดยปิดบังอักขระที่ถูกจับคู่ไปแล้ว ให้คะแนน mark ที่ตรงกัน และจองช่วงนั้นไว้ เพื่อไม่ให้ช่วงเดียวของ output ให้คะแนนสองบรรทัด บรรทัดที่เหลือถูกจัดแนวแบบ global ตามลำดับของเฉลยกับ output ที่ตัดอักขระที่จองไว้ออกทั้งหมด หากไม่มีขั้นนี้ mark ในบรรทัดที่อ่านคลาดเคลื่อนจะหลุดไป และเวอร์ชันแรกของเราไม่ผ่านการตรวจกับโมเดลตั้งต้น Recall คือ mark ที่ได้คะแนนหารด้วย mark ในเฉลย ส่วน precision คือ mark ที่ได้คะแนนหารด้วย mark *ทั้งหมด* ใน output จึงหักคะแนนข้อความที่เขียนเกิน F1 คิดแบบ micro-average ทุกข้อ

แบบกันการวนซ้ำ ในแบบนี้ คะแนนคิดจาก output ที่ย่อทุกช่วงที่วนซ้ำให้เหลือหนึ่งสำเนา ช่วงที่วนซ้ำคือหน่วยยาว 4–400 ตัวอักษรที่มีตัวอักษร ซ้ำกันเป๊ะอย่างน้อยแปดครั้ง (หกครั้งสำหรับหน่วยที่ยาว 50 ตัวอักษรขึ้นไป) ส่วน precision ยังนับ mark ทุกตัวของ output เต็ม สำเนาที่ซ้ำจึงถูกหักคะแนนแต่ไม่ได้ recall แบบนี้เพิ่มขึ้นหลังจากผู้ร่วมงานสังเกตว่า alignment จับคู่สำเนาของหน่วยเดียวที่ซ้ำ 434 ครั้งเข้ากับพยางค์ทั่วไปในเฉลย ถ้าไม่มีการวนซ้ำ คะแนนทั้งสองแบบเท่ากัน

การตรวจสอบ การอ่านหน้าเดียวกันสองรอบต่อกันได้คะแนนต่ำกว่าการอ่านรอบเดียวที่ดีกว่ามาก recall ของโมเดลตั้งต้นต่างจาก global alignment ไม่เกิน 0.3 จุดในทุกเซลล์ และ recall ของ Typhoon บนหน้าเต็มเพิ่มขึ้น 1.6–1.8 จุดเมื่อไม่หักเรื่องลำดับ การหักคะแนนเรื่องลำดับอย่างเดียวทำให้ mark F1 ของ Typhoon บนหน้าเต็มลดจาก 94.9 เหลือ 93.1 (BQ; ส่วน TC จาก 95.3 เหลือ 93.7 โดย BQ และ TC นิยามในหัวข้อถัดไป) ส่วนโมเดลตั้งต้นแทบไม่ได้รับผลกระทบ

ตารางที่ 1 Checkpoint ที่ระบุ revision และค่าที่ใช้ในการรัน

รายการ	ค่า
Benchmark	ThaiOCRBench, ชุด test
ฉบับ (revision) ของ benchmark	`ca610d1ab330`
ชุด calibration	seed 20260927, 30%
Typhoon OCR 1.5	`9c8a8fa14905`
Qwen3-VL-2B / -4B	`89644892e4d8` / `ebb281ec`
PaddleOCR-VL-1.6	`c5630aba`
Wayu	`af0204b4`
การ decode	greedy, token ใหม่ 3,072 ตัว, fp16
ฮาร์ดแวร์	Kaggle Tesla T4, batch size 1

4. Experimental Setup

ข้อมูล เราใช้ชุด test ของ ThaiOCRBench [1] สองงาน คืองานอ่านทั้งหน้าและงานอ่านข้อความสั้น ชุด calibration สุ่มแบบกำหนด seed ไว้ 30% แบ่งชั้นตามงานและหมวด มีงานอ่านทั้งหน้า 69 ข้อ และงานอ่านข้อความสั้น 109 ข้อ ชุด locked ซึ่งเป็นส่วนที่เหลือ 70% ยังไม่ถูกเปิด เรารายงาน output และคะแนนรายข้อ โดยไม่เผยแพร่ภาพของ benchmark

โมเดล เราประเมิน Typhoon OCR 1.5 (2B) และโมเดลตั้งต้น Qwen3-VL-2B, Qwen3-VL-4B และโมเดลขนาด 0.9B ที่ออกแบบให้อ่านทีละกรอบข้อความ คือ PaddleOCR-VL-1.6 และ wayu-paxa-ocr-zero (เรียกว่า “Wayu”) ซึ่งเรานำมาอ่านทั้งหน้าด้วย นอกเหนือจากที่ออกแบบไว้ ทุก checkpoint ระบุ revision ไว้ (ตารางที่ 1) และเราใช้ revision ของ Typhoon ที่เผยแพร่ภายใต้ Apache-2.0 อย่างเดียว

Prompt และการ decode BQ คือคำถามของ benchmark ในแต่ละข้อ **TC** คือ prompt ที่ model card ของ Typhoon แนะนำ ส่วน Paddle และ Wayu ใช้ prompt `OCR:` การ decode ใช้ greedy สร้าง token ใหม่ไม่เกิน 3,072 ตัว ด้วย fp16 บน GPU Kaggle T4 แต่ละผลมาจากการรันครั้งเดียว output ถูกแปลงเป็นข้อความธรรมดาก่อนให้คะแนน BQ เป็น prompt หลัก และ TC เป็น prompt รอง

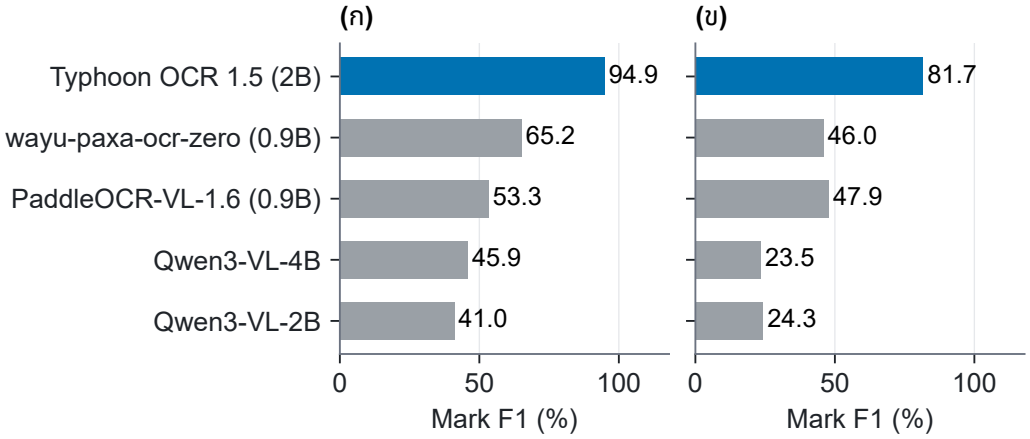
สถิติ ช่วงความเชื่อมั่นคือ bootstrap 95% ระดับรายข้อ ผลทั้งหมดเป็นผลเบื้องต้นบนชุด calibration และกฎ decode บางข้อถูกกำหนดหลังจากเห็นชุดนี้แล้ว ซึ่งเราระบุไว้เป็นรายข้อ

5. Where Marks Are Lost

5.1 Five Models

Typhoon ได้ mark F1 94.9 บนหน้าเต็ม และ 81.7 บนงานอ่านข้อความสั้น สูงกว่า checkpoint อีกสี่ตัวมาก (รูปที่ 1 และตารางที่ 2) เมื่อเทียบกับ Typhoon แล้ว Wayu ต่ำกว่า 29.6 จุด [39.9, 19.2] บนหน้าเต็ม การเพิ่มขนาดโมเดลตั้งต้นเป็นสองเท่าเปลี่ยน F1 บนหน้าเต็มเพียง +5.0 [−4.0, +14.1] และอัตราผิดของวรรณยุกต์ยังอยู่ราว 9.4–9.9% ค่ามัธยฐานของ character error rate ต่อหน้าของ Typhoon คือ 2.8% โมเดลทั้งหมดถูกเทียบตามที่เผยแพร่ ซึ่งงบ visual token ไม่เท่ากัน Typhoon ดีที่สุดในบรรดาโมเดลที่เราวัด ส่วน Pathumma-llm-vision 3.0 รายงานคะแนน ThaiOCRBench สูงกว่าในงานอ่านข้อความสั้นและคะแนนรวม [3] ซึ่งเรายังไม่ได้วัด **[Pathumma-llm-vision 3.0 และโมเดลอื่น ๆ]**

สำเนาของการวนซ้ำเคยได้ recall คะแนนแบบกันการวนซ้ำเปลี่ยน Typhoon ไม่เกิน 0.2 จุด แต่ทำให้



รูปที่ 1 Mark F1 แบบไม่หักเรื่องลำดับบนชุด calibration (prompt BQ; `OCR`: สำหรับโมเดล 0.9B สองตัว) Typhoon สูงสุดทั้งสองงาน

ตารางที่ 2 Mark F1, อัตราผิดของวรรณยุกต์หลังอ่านพยัญชนะถูก และสัดส่วน output ที่ชนขีดจำกัด token (%)

	อ่านทั้งหน้า			อ่านข้อความสั้น	
โมเดล	F1	ผิดวรรณยุกต์	ชนขีดจำกัด	F1	ชนขีดจำกัด
Typhoon OCR 1.5	**94.9**	0.65	2.9	**81.7**	0.9
Wayu	65.2	5.27	26.1	46.0	20.2
PaddleOCR-VL-1.6	53.3	14.57	39.1	47.9	14.7
Qwen3-VL-4B	45.9	9.89	27.5	23.5	11.9
Qwen3-VL-2B	41.0	9.41	43.5	24.3	14.7

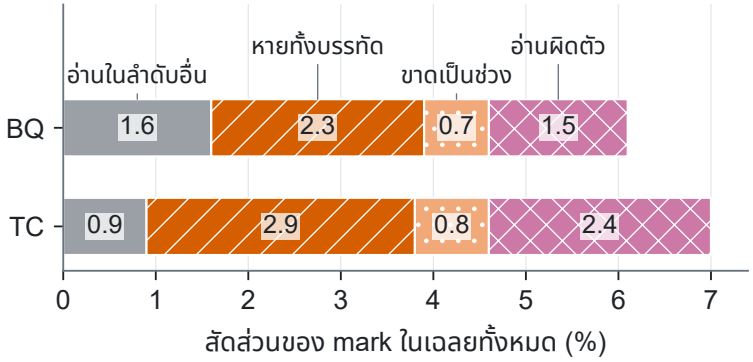
F1 บนหน้าเต็ม (BQ) ของโมเดลตั้งต้นลดจาก 41.0 เหลือ 34.4 และ recall ลดจาก 65.0 เหลือ 54.6 (ตารางที่ 3) กล่าวคือ ราวหนึ่งในหกของ mark ที่คะแนนแบบไม่หักลำดับให้แก่โมเดลมาจากสำเนาของการวนซ้ำ เราเปิดดู output ของทุกช่วงที่ถูกตัดแล้ว ทั้ง 126 ช่วงชนขีดจำกัด token ช่วงที่สั้นที่สุดมี 15 สำเนา และไม่มีข้อความปกติถูกตัดออก

5.2 Typhoon's Lost Marks Are Mostly Not Read

บนหน้าเต็ม Typhoon ทำ mark ผิด 6.2% (BQ) และ 7.1% (TC) ของ mark ในเฉลยทั้งหมด เราจัดแต่ละข้อผิดเข้าสาเหตุเดียวโดยอัตโนมัติ (รูปที่ 2) ได้แก่ บรรทัดถูกอ่านในลำดับอื่น (มีช่วงของ output ตรงกับบรรทัดนั้นด้วย character error rate ต่ำกว่า 20% สำหรับบรรทัดที่ยาวอย่างน้อยแปดตัวอักษร) บรรทัดหายไปทั้งบรรทัด ข้อความขาดเป็นช่วงภายในบรรทัดที่เก็บไว้ หรืออ่าน mark ผิดตัว บรรทัดที่หายทั้งบรรทัดคิดเป็น 2.3% (BQ) และ 2.9% (TC) ของ mark ข้อความที่ขาดเป็นช่วงคิดเป็น 0.7% และ 0.8% และการอ่านผิดตัวคิดเป็น 1.5% และ 2.4% ส่วนลำดับการอ่านอย่างเดียวคิดเป็น 1.6% และ 0.9% ซึ่งไม่ใช่ความผิดพลาดของการอ่าน เมื่อเปิดดู output จริง บรรทัดที่หายส่วนใหญ่เป็นข้อความใน infographic และกราฟ โฆษณาที่แทรกอยู่ รวมถึงชื่อเรื่องและส่วนประกอบของหน้า ซึ่ง Typhoon เขียนเป็นตัวแทนภาพไว้ สอดคล้องกับที่ผู้พัฒนาระบุเรื่อง infographic [2] ส่วนโมเดลตั้งต้นสูญเสีย mark ราว 41% การจัดสาเหตุเป็นแบบอัตโนมัติ และเราตรวจภาพหน้าเอกสารเฉพาะหกหน้าที่มี mark ที่หายเป็นส่วนใหญ่ **[การจัดสาเหตุสำหรับโมเดลอื่น]**

ตารางที่ 3 Recall (R), precision (P) และ F1 (%) เมื่อให้และไม่ให้คะแนนแก่สำเนาของการวนซ้ำ หน้าเต็ม BQ

	แบบไม่หักลำดับ			แบบกันการวนซ้ำ		
โมเดล	R	P	F1	R	P	F1
Typhoon	96.1	93.7	94.9	96.1	93.6	94.8
Qwen3-VL-2B	65.0	29.9	41.0	54.6	25.1	34.4



รูปที่ 2 mark ที่ Typhoon สูญเสียบนหน้าเต็มไปกับสาเหตุใด คิดเป็นสัดส่วนของ mark ในเฉลยทั้งหมด

5.3 Given the Right Context, the Prior Is Already Right

ในการทดสอบแบบเลือกคำตอบบังคับ (forced choice) เราป้อนข้อความที่ถูกต้องจนถึงตำแหน่งของ mark แล้วให้โมเดลให้คะแนนตัวเลือกที่เป็นไปได้ของ mark นั้นทั้งแบบมีภาพและไม่มีภาพ (fp32, tokenize ในบริบทเดียวกัน) Typhoon เลือกวรรณยุกต์ถูกบนหน้าเต็ม 99.7% เมื่อมีภาพ และ 98.8% เมื่อไม่มีภาพ ส่วนโมเดลตั้งต้นได้ 98.1% และ 98.9% (รูปที่ 3 ด้านซ้าย) ภาพช่วยเพิ่ม 0.9 จุดให้ Typhoon และลดลง 0.8 จุดสำหรับโมเดลตั้งต้น การทดสอบนี้เป็นการเลือกตอบแบบบังคับภายใต้บริบทที่ถูกต้อง ไม่ใช่การ generate จริง ในคู่โมเดลนี้เพียงคู่เดียว โมเดลที่ train เฉพาะทางได้ประโยชน์จากภาพมากกว่าโมเดลตั้งต้น ไม่ใช่น้อยกว่า และเราไม่ขยายความไปเกินกว่านี้ **[การทดสอบเดียวกันกับ Pathumma และ Qwen3-VL-4B]**

ความมั่นใจของโมเดล ความน่าจะเป็นของ token ที่ Typhoon เขียนเองบอกตำแหน่งที่อ่าน mark ผิดได้ ค่าความน่าจะเป็นต่ำสุดของ token ภายในกลุ่มอักขระแยก mark ที่อ่านผิดออกจาก mark ที่ถูกด้วย AUROC 0.918 (BQ) และ 0.935 (TC) บน output แบบ greedy ของโมเดลเอง

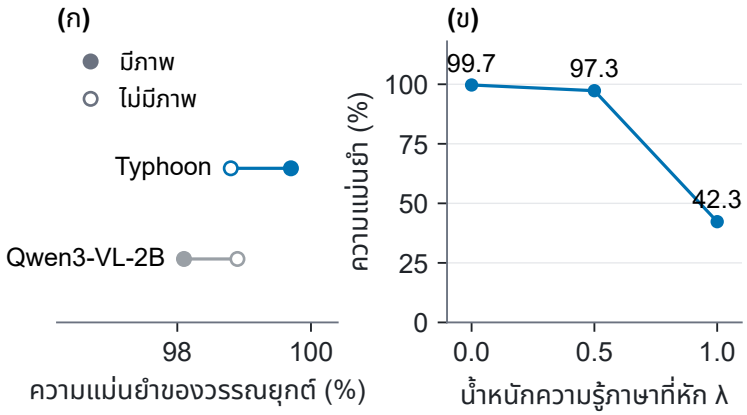
6. Training-Free Decoding and Marks

6.1 Loops Are Rare and Costly

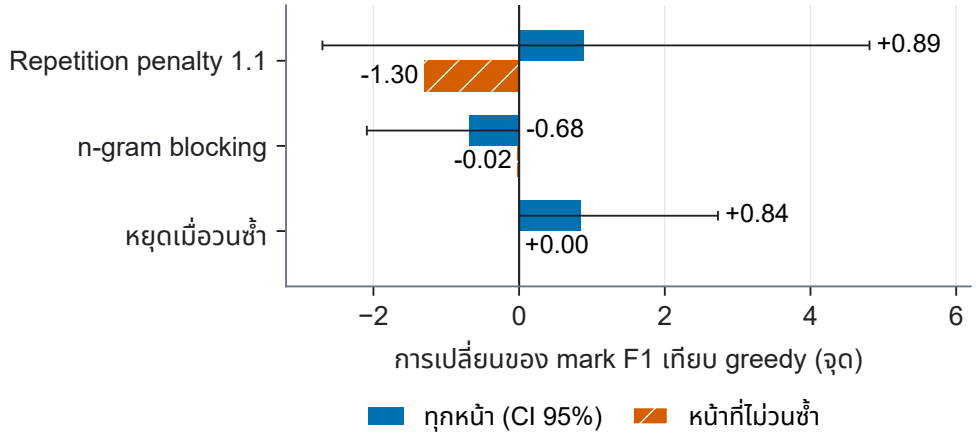
จาก output ของ Typhoon 356 รายการ (BQ และ TC ทั้งสองงาน) มี 11 รายการที่วนซ้ำ หน้าที่วนซ้ำเพียง 2–5 หน้าครอบครอง 68–76% ของ mark ที่เขียนเกินทั้งหมดในแต่ละเซลล์ ถ้าไม่มีหน้าเหล่านี้ F1 บนหน้าเต็ม (BQ) จะเป็น 97.4 แทน 94.9 ค่านี้คือเพดานที่ได้จากการตัดหน้าเหล่านั้นออก ไม่ใช่ผลของวิธีใด

6.2 Three Ways to Handle a Loop

เราเทียบวิธีจัดการการวนซ้ำสามวิธีของ Typhoon กับ greedy decoding (รูปที่ 4 หน้าเต็ม BQ)



รูปที่ 3 ซ้าย: ความแม่นยำของวรรณยุกต์ที่ตำแหน่งที่ลงทะเบียนไว้ เมื่อมีและไม่มีภาพ ขวา: ความแม่นยำของ Typhoon เมื่อหักคะแนนแบบไม่มีภาพด้วยน้ำหนัก λ



รูปที่ 4 การเปลี่ยนแปลงของ mark F1 บนหน้าเต็มเทียบกับ greedy decoding (Typhoon, BQ) แยกตามทุกหน้าและเฉพาะหน้าที่ไม่วนซ้ำ

- **Repetition penalty 1.1** ซึ่งเป็นค่าต้านการวนซ้ำใน package ของ Typhoon เอง ใช้ penalty แบบเดียวกับ CTRL [18]
- **n-gram blocking** แบบเดียวกับโค้ดของ DeepSeek-OCR คือห้าม 30-gram ซ้ำภายใน 90 token
- **หยุดเมื่อตรวจพบการวนซ้ำ** เมื่อหน่วยข้อความซ้ำแปดครั้ง (หกครั้งสำหรับหน่วยที่ยาว 50 ตัวอักษรขึ้นไป) การ decode หยุดที่สำเนาแรก greedy decoding กำหนดผลล่วงหน้าตาม prefix เราจึงคำนวณจุดตัดแบบ offline จาก output greedy ที่ครบแล้ว ส่วนการทำตอน decode จริง เรายังไม่ได้ตรวจเทียบกับผลนี้

Repetition penalty เปลี่ยน output เกือบทุกรายการ (หน้าเต็ม BQ ที่ไม่วนซ้ำไม่มีหน้าใดเหมือน greedy เลย) และทำให้ F1 ของหน้าที่ไม่วนซ้ำลดลง 1.1–1.3 จุด ในงานอ่านข้อความสั้น (BQ) อัตราผิดของวรรณยุกต์เพิ่มจาก 17.1% เป็น 22.6% ส่วนบนทุกหน้า ผลแยกจากศูนย์ไม่ได้ (+0.89 [−2.70, +4.81]) ข้อความไทยมีพยางค์และวรรณยุกต์ซ้ำกันเป็นปกติ ซึ่ง penalty ทั้งหน้าอาจกดไว้ แต่เรายังไม่ได้ทดสอบคำอธิบายนี้

n-gram blocking ปลอดภัยเมื่อไม่มีการวนซ้ำ (output ที่ไม่วนซ้ำไม่เปลี่ยน 92–100%, ΔF1 $\leq$ 0.05) และหยุดการวนซ้ำได้ 6 จาก 11 รายการ ในรายการที่หยุดไม่ได้ โมเดลหนีไปวนแบบใกล้เคียงของเดิม เช่น ชื่อซอยที่เพี้ยนไปเป็นชื่อซอยคล้ายกันสามชื่อ ข้อความเหล่านี้มี mark เพิ่ม ทำให้ mark ที่เขียนเกินสูงขึ้น และ F1

ตารางที่ 4 Mark F1 ก่อนและหลังหยุดเมื่อตรวจพบการวนซ้ำ (แบบ offline)

	อ่านทั้งหน้า		อ่านข้อความสั้น	
โมเดล	greedy	+ หยุด	greedy	+ หยุด
Typhoon (BQ)	94.9	95.7	81.7	85.0
Typhoon (TC)	95.3	96.0	75.8	78.2
Wayu	65.2	76.0	46.0	70.4

[โมเดลอื่น]

บนหน้าเต็มต่ำลง 0.68 (BQ) เราตีความว่าการวนซ้ำเป็นสถานะดึงดูดของ decoder การห้ามสำเนาที่ซ้ำตรงตัวจึงย้ายโมเดลไปอยู่ในสถานะข้างเคียง ซึ่งเป็นการตีความ ไม่ใช่กลไกที่ทดสอบแล้ว

หยุดเมื่อพบการวนซ้ำ ไม่เปลี่ยน output ที่ไม่วนซ้ำเลย (ไม่มี output ที่ไม่วนซ้ำถูกเปลี่ยนในเซลล์ใด) และเพิ่ม mark F1 บนหน้าเต็ม 0.84 (BQ) และ 0.67 (TC) และเพิ่ม F1 งานอ่านข้อความสั้น 3.30 และ 2.39 ทั้งยังลด token ที่สร้างลง 11.2% recall ลดลงไม่เกิน 0.16 จุด เพราะการหยุดไม่เคยกู้ข้อความที่การวนซ้ำทำให้ข้ามไป กฎนี้เขียนหลังจากเห็นผลการเปรียบเทียบก่อนหน้า และในแต่ละเซลล์มี output ที่วนซ้ำเพียงหนึ่งถึงสี่รายการ ช่วงความเชื่อมั่นทุกช่วงจึงรวมศูนย์ไว้

6.3 The Stop on a Second Model

Wayu วนซ้ำบ่อยกว่ามาก คือ 26.1% ของ output บนหน้าเต็มชนขีดจำกัด token เมื่อใช้การหยุดแบบ offline เดียวกันกับ output greedy ของ Wayu mark F1 บนหน้าเต็มเพิ่มจาก 65.2 เป็น 76.0 (+10.8 [5.3, 16.1]) และ F1 งานอ่านข้อความสั้นเพิ่มจาก 46.0 เป็น 70.4 (+24.4 [10.7, 35.7]) (ตารางที่ 4) ผลที่เพิ่มอยู่ที่ precision (+28.3 บนหน้าเต็ม) โดยแลกกับ recall ที่ลดลง 3.3 จุด repetition penalty 1.05 อย่างเดียว ซึ่งเป็นค่าในตัวอย่างของ Wayu เอง ไม่ช่วย (+2.8 [−3.6, +9.6] บนหน้าเต็ม และ −11.3 [−22.5, +4.5] บนงานอ่านข้อความสั้น) การหยุดลด token ที่สร้างลง 34% บนหน้าเต็ม ในการรันนี้ 72% ของช่องว่าง recall ระหว่าง Wayu กับ Typhoon บนหน้าเต็มอยู่ใน 18 หน้าที่ Wayu วนซ้ำ และไม่มีวิธี decode ใดที่ทดสอบกู้คืนได้ เรายังแยกไม่ได้ว่าการวนซ้ำเป็นสาเหตุหรืออาการ

6.4 Subtracting the Language Prior Removes Correct Marks

ในการทดสอบแบบเลือกตอบบังคับ การหักคะแนนแบบไม่มีภาพด้วยน้ำหนัก λ ทำให้ความแม่นยำของวรรณยุกต์ของ Typhoon ลดจาก 99.7% เป็น 97.3% ที่ $\lambda = 0.5$ และเหลือ 42.3% ที่ $\lambda = 1$ (รูปที่ 3 ด้านขวา) ส่วนโมเดลตั้งต้นลดเหลือ 84.7% ที่ 0.5 ในการ generate จริง บนกลุ่มย่อย 24 ข้อ contrast แบบ M3ID-form ของเราหยุดการวนซ้ำของโมเดลตั้งต้นได้ (character error rate จาก 1.12 เหลือ 0.21) แต่อัตราผิดของวรรณยุกต์ในข้อที่เก็บไว้เพิ่มจาก 3.7% เป็น 7.6% ส่วนใหญ่เพราะวรรณยุกต์ถูกลบ (21 เป็น 55 ใน 12 ข้อ) ส่วน Typhoon เปลี่ยนเพียงเล็กน้อยและกระจุกอยู่ในไม่กี่ข้อ ผลทั้งสองชี้ไปทางเดียวกัน คือความรู้ภาษาเกี่ยวกับวรรณยุกต์ถูกอยู่แล้ว การหักออกจึงลบวรรณยุกต์ที่ถูกไปด้วย

6.5 Speed

Speculative decoding แบบ prompt lookup ทำให้ Typhoon เร็วขึ้น 1.14–1.15 เท่า (โมเดลตั้งต้น 1.16–1.21 เท่า) บนหน้าที่ไม่เสื่อมเป็นการวนซ้ำ โดย output เหมือนเดิมยกเว้นจุดที่คะแนนสองตัวเลือกเกือบเท่ากันใน fp16

(88–93% ของทุกข้อเหมือนเดิม) ตัวอ้างอิง greedy ให้ output ตรงกับ output หลักของเราทุกตัวอักษรในทั้ง 177 ข้อที่จับเวลา **[ความเร็วเมื่อใช้การหยุดร่วมกับ speculative decoding]**

7. Discussion and Limitations

สิ่งที่สรุปได้ สำหรับโมเดลที่ศึกษา ความผิดของ mark ในโมเดล OCR ภาษาไทยที่เก่งอยู่แล้วส่วนใหญ่เกิดก่อนถึงตัว mark ได้แก่ บรรทัดที่ไม่ได้อ่าน ข้อความที่ขาดเป็นช่วง และการวนซ้ำ เมื่อได้บริบทที่ถูก วรรณยุกต์เกือบแน่นอนอยู่แล้ว และวิธีที่หักความรู้ภาษาออกต้องแลกด้วยวรรณยุกต์ ส่วนวิธีที่ถูกและปลอดภัยในการจัดการการวนซ้ำคือหยุดมัน

ข้อจำกัด (1) benchmark เดียวและสองงาน (2) ทุกผลอยู่บนชุด calibration และกฎการหยุด คะแนนแบบกันการวนซ้ำ และการให้คะแนนถูกเขียนหลังจากเห็นข้อมูล ส่วนชุด locked ยังไม่ถูกเปิด **[รอดำเนินการ]** (3) โมเดลถูกเทียบตามที่เผยแพร่ งบ visual token ไม่เท่ากัน และ Paddle กับ Wayu เป็นตัวอ่านทีละกรอบที่นำมาอ่านทั้งหน้านอกเหนือจากที่ออกแบบ (4) output ที่วนซ้ำมีน้อย ช่วงความเชื่อมั่นของ Typhoon ส่วนใหญ่จึงรวมศูนย์ไว้ ข้อสรุปจึงเป็นเรื่องกลไก ไม่ใช่อัตรา (5) การจัดสาเหตุเป็นแบบอัตโนมัติ (6) การทดสอบแบบเลือกตอบบังคับใช้บริบทที่ถูกต้อง (7) contrast ที่เราใช้เป็นการเขียนใหม่และทดสอบบน 24 ข้อ (8) เงื่อนไขการใช้ Typhoon: เราใช้ revision ที่เผยแพร่ภายใต้ Apache-2.0 อย่างเดียว ขณะที่เงื่อนไขปัจจุบันของผู้พัฒนาขอให้ได้รับความยินยอมเป็นลายลักษณ์อักษรก่อนนำไปเทียบประสิทธิภาพ

8. Conclusion

เครื่องหมายสระและวรรณยุกต์ไม่ใช่จุดที่ OCR แบบเปิดที่เก่งที่สุดพลาด มันพลาดก่อนหน้านั้น คือไม่ได้อ่าน หาไม่เจอ หรือวนซ้ำ การหยุดเมื่อตรวจพบการวนซ้ำเป็นขั้นตอนที่ปลอดภัย ไม่ต้อง train เพิ่ม และได้ผลที่วัดได้ ส่วนการหักความรู้ภาษาออกไม่ใช่ **[ข้อสรุปสุดท้ายรอผลการทดลองที่เหลือ]**

Ethical and Licensing Notes

โมเดลทั้งหมดเป็นโมเดลแบบเปิดและใช้ตามสัญญาอนุญาตของแต่ละตัว (Apache-2.0 หรือ MIT โดย Typhoon ใช้ revision ที่เป็น Apache-2.0 อย่างเดียว) ThaiOCRBench เผยแพร่ภายใต้ CC-BY-SA-4.0 เราอ้างอิงบทความและ revision ของ benchmark รายงาน output และคะแนน และไม่เผยแพร่ภาพของ benchmark ซ้ำ

เอกสารอ้างอิง

[1] S. Nonesung, T. Jaknamon, S. Chaiophat, N. Nitarach, C. Wittayasakpan, W. Sirichotedumrong, A. Na-Thalang, and K. Pipatanakul, “ThaiOCRBench: A task-diverse benchmark for vision-language understanding in Thai,” in *Proceedings of IJCNLP-AACL 2025 (Long Papers)*, 2025, pp. 1653–1675, arXiv:2511.04479. [Online]. Available: https://aclanthology.org/2025.ijcnlp-long.89

[2] S. Nonesung, N. Nitarach, T. Jaknamon, P. Taveekitworachai, and K. Pipatanakul, “Typhoon OCR: Open vision-language model for Thai document extraction,” 2026. [Online]. Available: https://arxiv.org/abs/2601.14722

[3] NECTEC, "Pathumma-llm-vision-3.0.0-re," Hugging Face model card, 2026, revision 08bb11dcabf509ac3fa3ac0014ecd44eadb9729b. [Online]. Available: https://huggingface.co/nectec/Pathumma-llm-vision-3.0.0-re

[4] K. Pipatanakul, "How far can synthetic data take Thai OCR?" 2026. [Online]. Available: https://arxiv.org/abs/2609.03595

[5] S. Leng, H. Zhang, G. Chen, X. Li, S. Lu, C. Miao, and L. Bing, "Mitigating object hallucinations in large vision-language models through visual contrastive decoding," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 13 872–13 882.

[6] A. Favero, L. Zancato, M. Trager, S. Choudhary, P. Perera, A. Achille, A. Swaminathan, and S. Soatto, "Multi-modal hallucination control by visual information grounding," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, arXiv:2403.14003.

[7] S. Bai, Y. Cai, R. Chen *et al.*, "Qwen3-VL technical report," 2025. [Online]. Available: https://arxiv.org/abs/2511.21631

[8] C. Cui, T. Sun, S. Liang *et al.*, "PaddleOCR-VL: Boosting multilingual document parsing via a 0.9B ultra-compact vision-language model," 2025. [Online]. Available: https://arxiv.org/abs/2510.14528

[9] T. Do, D. P. Tran, A. Vo, and D. Kim, "Reference-based post-OCR processing with LLM for precise diacritic text in historical document recognition," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 27, 2025, pp. 27 951–27 959.

[10] H. Elsafty, F. Aldabbas, and R. Sifa, "Arabic OCR in the age of multimodal models: A comprehensive comparative evaluation," in *2025 IEEE International Conference on Big Data*, 2025, pp. 5437–5442, lamarr Institute publication page.

[11] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *International Conference on Learning Representations (ICLR)*, 2020, arXiv:1904.09751.

[12] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2308.13418.

[13] J. Poznanski, A. Rangapur, J. Borchardt, J. Dunkelberger, R. Huff, D. Lin, C. Wilhelm, K. Lo, and L. Soldaini, "olmOCR: Unlocking trillions of tokens in PDFs with vision language models," 2025. [Online]. Available: https://arxiv.org/abs/2502.18443

[14] PaddlePaddle, "PaddleX PaddleOCR-VL pipeline: `truncate_repetitive_content`," GitHub source code, 2025. [Online]. Available: https://github.com/PaddlePaddle/PaddleX/blob/ffb64904d23708863ff5b8da312a5cbd52a7f462/paddlex/inference/pipelines/paddleocr_vl/uilts.py

[15] DeepSeek-AI, "DeepSeek-OCR source code: `NoRepeatNGramLogitsProcessor`," GitHub, 2025. [Online]. Available: https://github.com/deepseek-ai/DeepSeek-OCR

[16] H. Wei, Y. Sun, and Y. Li, "DeepSeek-OCR: Contexts optical compression," 2025. [Online]. Available: https://arxiv.org/abs/2510.18234

[17] Z. Yang, J. Tang, Z. Li, P. Wang, J. Wan, H. Zhong, X. Liu, M. Yang, P. Wang, S. Bai, L. Jin, and J. Lin, "CC-OCR: A comprehensive and challenging OCR benchmark for evaluating large multimodal models in literacy," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 21 744–21 754.

[18] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," 2019.

[19] S. Liu, K. Zheng, and W. Chen, "Paying more attention to images: A training-free method for alleviating hallucination in LVLMs," in *European Conference on Computer Vision (ECCV)*, 2024.

[20] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning (ICML)*, 2023, arXiv:2211.17192.

[21] A. Saxena, "Prompt lookup decoding," GitHub, 2023. [Online]. Available: https://github.com/apoorvumang/prompt-lookup-decoding